

Jenkins is a powerful tool that simplifies the software delivery process. It's a flexible platform used for a variety of software engineering tasks, but its main focus is managing CI/CD pipelines. Jenkins ensures that code changes are tested, validated, and deployed efficiently.

In this article, we'll dive into the key features, benefits, and real-world uses of Jenkins in modern software development. We'll also explore how it plays a crucial role in DevOps practices.

What is Jenkins?

- Jenkins is an open-source automation tool designed for Continuous Integration and Continuous Delivery (CI/CD).
- It automates tasks like building, testing, and deploying code, triggered by events such as commits, branch updates, or pull requests.
- Jenkins supports different environments, making it highly versatile for various development workflows.
- Its extensive plugin ecosystem allows easy integration with a wide range of tools and technologies.
- Built with Java, Jenkins works seamlessly across multiple operating systems, including Windows, Linux, and macOS.

What is Jenkins Used For?

- **Build Automation:** Jenkins automates the building of software projects, ensuring consistent and reliable results.
- **Testing:** It integrates with testing tools to automatically execute tests and validate code changes.
- **Deployment:** Jenkins simplifies the deployment of applications across different environments.
- **CI/CD Pipelines:** It orchestrates continuous integration and delivery pipelines, streamlining the software delivery process.
- **Workflow Management:** Jenkins handles complex workflows across multiple stages of development.
- **Tool Integration:** It integrates seamlessly with tools like Git, Maven, Docker, and Kubernetes, covering all phases of the software development lifecycle.
- **Monitoring and Reporting:** Jenkins monitors builds and deployments, providing detailed reports on successes, failures, and performance metrics.

Is Jenkins a CI or CD Tool?

Jenkins is primarily a **Continuous Integration (CI)** tool but is also widely used for **Continuous Delivery (CD)**.

- **Continuous Integration (CI):**

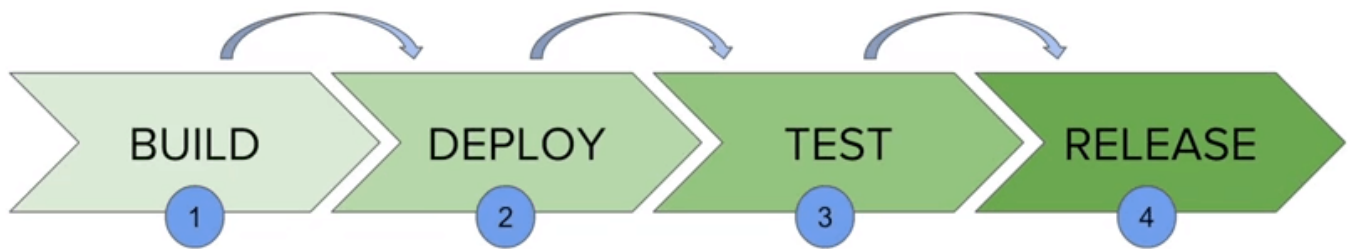
- CI focuses on automating the process of integrating code changes into a shared repository.
- Jenkins helps development teams merge and test code in the version control system, triggering pipelines that build and run test cases automatically.
- The main goal of CI is to provide developers with quick feedback to identify and fix bugs early, reducing risks before deploying to production.

- **Continuous Delivery (CD):**

- CD ensures that software releases are prepared in a reliable and automated way, ready to be deployed into higher environments.
- Jenkins supports CD by creating deployment artifacts, managing dependencies, and configuring environment-specific variables.

- Unlike **Continuous Deployment**, CD typically involves a manual approval step before releasing builds to production, adding an extra layer of control.

How Jenkins Works in CI/CD Pipelines



- **Integration with Git:** Jenkins integrates with Git repositories where developers collaborate and commit code changes.

- It automatically detects changes in the repository and triggers the build process.
- The build process includes **compilation**, **testing**, and **error reporting**, providing quick feedback to developers.
- This ensures a faster development cycle without impacting the production environment.

- **Automation in Continuous Delivery:**

- Jenkins automates tasks like managing dependencies and creating deployment artifacts.
- Common artifacts include **JAR files** for Java applications and **container images** for containerized deployments.
- These artifacts are pushed to appropriate repositories, ready for deployment.

- **Robust Plugin Ecosystem:**

- Jenkins is open-source and extensible, supported by a large community.
- Its extensive plugin library enables integration with tools for version control, source code management, testing frameworks, deployment, reporting, and more.
- This modular design allows organizations to build flexible and highly customized pipelines tailored to their specific workflows.

Conclusion

Jenkins is a cornerstone tool in modern software development, empowering teams to automate and streamline their CI/CD workflows. By integrating with tools like Git and leveraging its extensive plugin ecosystem, Jenkins simplifies complex tasks such as building, testing, and deploying applications. Its flexibility, extensibility, and robust community support make it a reliable choice for organizations aiming to enhance their development pipelines. Whether it's reducing errors, providing faster feedback, or enabling seamless delivery, Jenkins plays a crucial role in accelerating the software development lifecycle and supporting DevOps practices.

Integrating Git with Jenkins: A Complete Guide

Introduction

Continuous Integration (CI) is a core practice of modern DevOps. Integrating version control systems like Git with Jenkins empowers teams to automate code **builds, tests & deployments** seamlessly. In this article, we'll explore how to integrate Git with Jenkins and make the things automation.

Why Integrate Git with Jenkins?

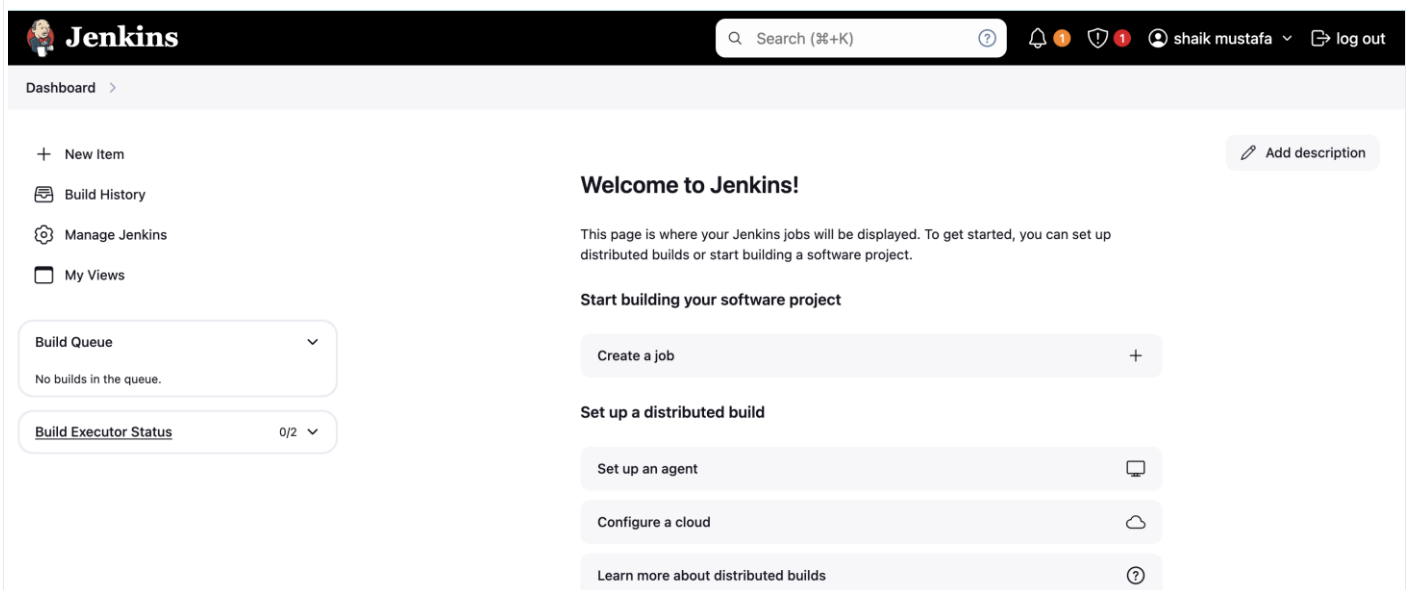
- **Automated Builds:** Automatically trigger builds whenever code changes are pushed to the repository.
- **Seamless Collaboration:** Boost team productivity by catching and resolving issues early in the development pipeline, ensuring smoother workflows and fewer delays.
- **Effortless Scalability:** Streamline processes for large-scale projects, making it easier to manage distributed teams and complex workflows efficiently.

Prerequisites

- **Jenkins installed** and configured on your server.
- **A Git repository** (GitHub or BitBucket).
- Necessary **Jenkins plugins** (e.g., Git Plugin).

Step-by-Step Guide

STEP-1: Install Jenkins and setup dashboard

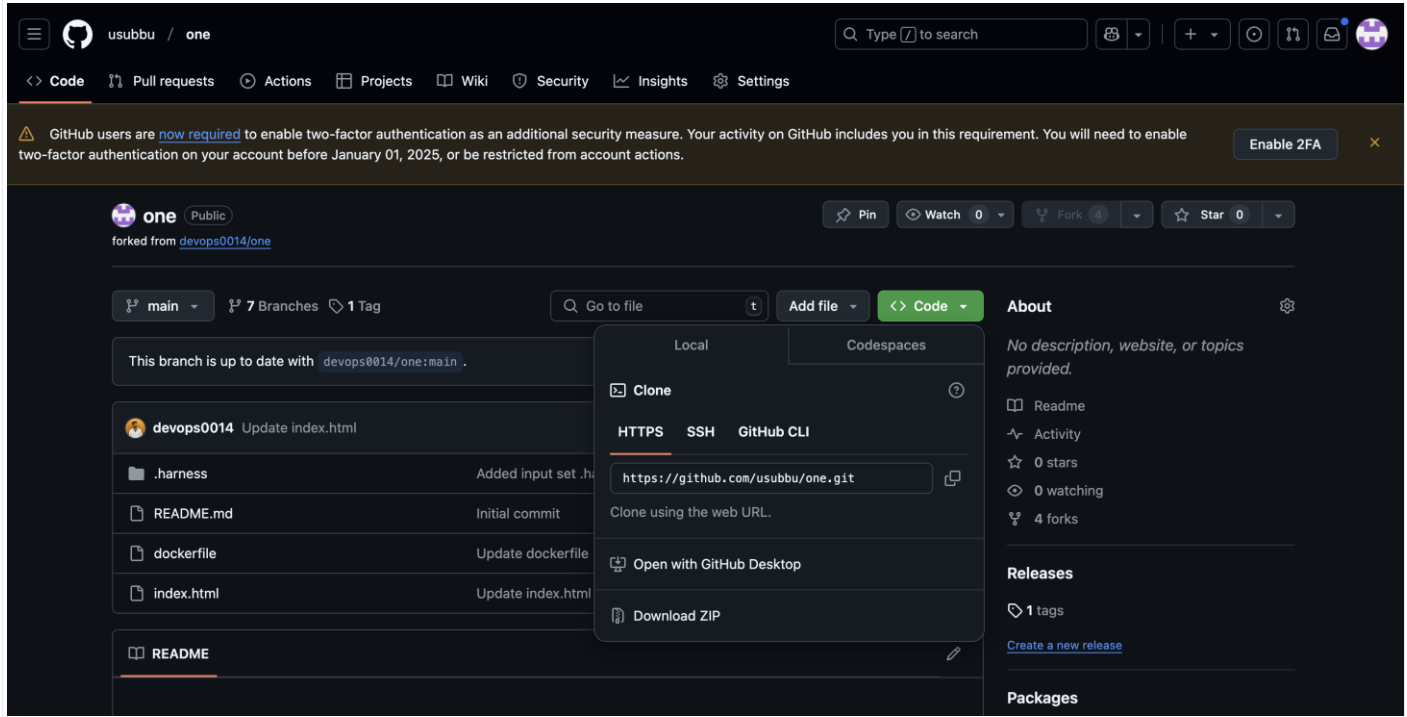


STEP-2: Install git on Jenkins server

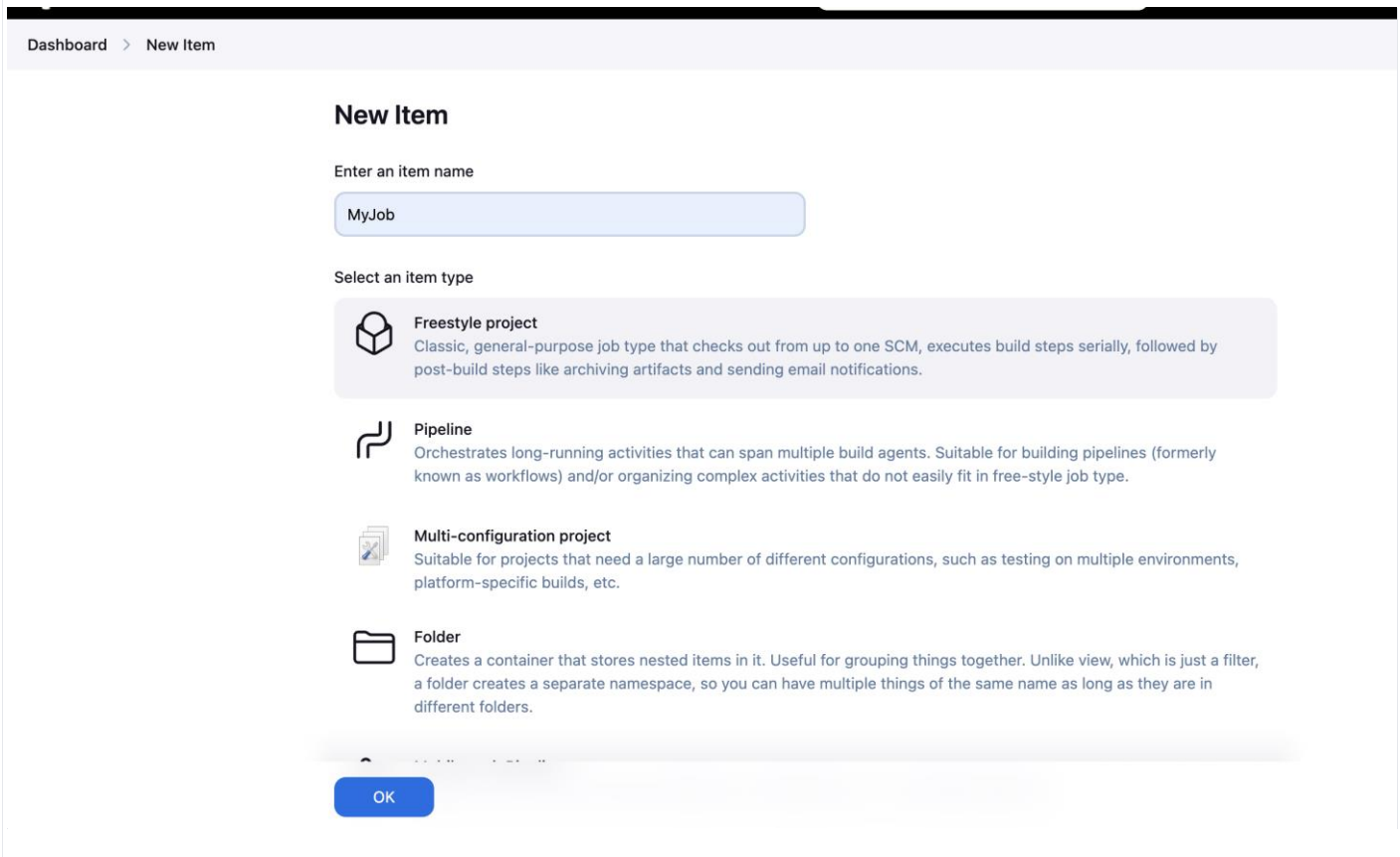
Copy
Copy

```
yum install git -y
```

STEP-3: Open your github repo and copy https link (<https://github.com/usubbu/one.git>)



STEP-4: Go to jenkins dashboard, **create a job**



Now go to **source code management** and paste your **repo-url** and add the **credentials**

Dashboard > MyJob > Configuration

Configure

- General
- Source Code Management**
- Build Triggers
- Build Environment
- Build Steps
- Post-build Actions

Source Code Management

☐ None
☒ Git ?

Repositories ?

Repository URL ?

https://github.com/usubbu/one.git

Credentials ?

- none -

+ Add

Jenkins

Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

Username : <your github user-name>

Password: <Enter Github Token>

ID: github

Jenkins Credentials Provider: Jenkins

Global credentials (unrestricted)

Kind

Username with password

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

Username ?

usubbu

☐ Treat username as secret ?

Password ?

.....

ID ?

github

and click on **add**

Source Code Management

☐ None

☒ Git ?

Repositories ?

Repository URL ?

https://github.com/usubbu/one.git

Credentials ?

usubbu/***** (github password)

+ Add

Advanced ▾

Add Repository

Now enter the branch name that you want to build. By default it will be **master**, if you wish to build other branch make sure to change the branch name that you want to build.

Branches to build ?

Branch Specifier (blank for 'any') ?

*/master

Add Branch

Now **save** the job and build it.



Status



Changes



Workspace



Build Now



Configure



Delete Project



Rename

MyJob

Permalinks

Builds



Today



#1 06:36



Hurray! Now we will get all the files from GitHub to CI Server. So developer make any changes in GitHub e have to build here in Jenkins.

Now lets make it automate - Whenever developer make some changes in GitHub, Jenkins has to trigger the build automatically. To implement it we can use the below concepts

1. Webhooks
2. Poll SCM
3. Build Periodically

What is a Webhook?

A **webhook** is a way for one application to send real-time data to another application whenever a specific event occurs. In the context of Git and Jenkins, webhooks enable Jenkins to receive notifications from a Git repository (e.g., GitHub, GitLab) whenever a commit is pushed or a pull request is created.

How to Integrate Webhooks with Jenkins

Step 1: Configure Jenkins Job

1. Go to the Jenkins dashboard and open the job configuration page.
2. Scroll to the **Build Triggers** section.
3. Select **GitHub hook trigger for GITScm polling** (if using GitHub).

Dashboard > MyJob > Configuration

Configure

General

Source Code Management

Build Triggers

Build Environment

Build Steps

Post-build Actions

Build Triggers

☐ Trigger builds remotely (e.g., from scripts) ?

☐ Build after other projects are built ?

☐ Build periodically ?

☒ GitHub hook trigger for GITScm polling ?

☐ Poll SCM ?

Build Environment

☐ Delete workspace before build starts

Now save the job

Step 2: Set Up Webhooks in the Git Repository

The process varies slightly depending on the Git platform (GitHub, GitLab, Bitbucket). Below is the guide for GitHub:

For GitHub:

1. Go to your GitHub repository.
2. Navigate to **Settings > Webhooks**.
3. Click **Add webhook**.
4. Fill out the form:

- **Payload URL:** Enter the Jenkins webhook URL. It typically looks like this:

Copy

Copy

`http://<JENKINS_URL>:<PORT>/github-webhook/`

Example: `http://192.168.1.100:8080/github-webhook/`

- **Content Type:** Select `application/json`.
- Click **Add webhook**.

This screenshot shows the GitHub 'Webhooks / Manage webhook' settings page. The left sidebar contains navigation links for General, Access, Collaborators, Moderation options, Code and automation, Branches, Tags, Rules, Actions, Webhooks (selected), Environments, Codespaces, Pages, Security, Code security, and Deploy keys. The main content area has two tabs: 'Settings' and 'Recent Deliveries'. The 'Settings' tab is active, showing fields for 'Payload URL' (http://52.90.173.232:8080/github-webhook/), 'Content type' (application/json), and 'Secret'. There is also an 'SSL verification' section with radio buttons for 'Enable SSL verification' (selected) and 'Disable (not recommended)'. A warning banner at the top states that GitHub users are now required to enable two-factor authentication.

This screenshot shows the GitHub 'Webhooks' settings page. The left sidebar is the same as the previous image. The main content area has a title 'Webhooks' and an 'Add webhook' button. Below the title, there is an explanatory text: 'Webhooks allow external services to be notified when certain events happen. When the specified events happen, we'll send a POST request to each of the URLs you provide. Learn more in our [Webhooks Guide](#).' A list of webhooks is shown, with one entry: 'http://54.234.94.233:8080/github... (push)'. Below the URL, it says 'Last delivery was successful.' There are 'Edit' and 'Delete' buttons next to the webhook entry.

Now lets make some changes on GitHub, Jenkins will automatically trigger the job.

 Status

 Changes

 Workspace

 Build Now

 Configure

 Delete Project

 GitHub Hook Log

 Rename

 MyJob

Permalinks

- [Last build \(#1\), 14 min ago](#)
- [Last stable build \(#1\), 14 min ago](#)
- [Last successful build \(#1\), 14 min ago](#)
- [Last completed build \(#1\), 14 min ago](#)

Builds ...

 Filter /

Pending

 #2 
| In the quiet period. Expires in 3.3 sec

Today

 #1 06:36 

 Status Changes Workspace Build Now Configure Delete Project GitHub Hook Log Rename MyJob

Permalinks

- [Last build \(#1\), 14 min ago](#)
- [Last stable build \(#1\), 14 min ago](#)
- [Last successful build \(#1\), 14 min ago](#)
- [Last completed build \(#1\), 14 min ago](#)

Builds

 Filter

Today



#2 07:20



#1 06:36



Benefits of Using Webhooks with Jenkins

- **Instant Build Triggers:** Eliminate delays caused by SCM polling.
- **Resource Efficiency:** Save CPU and bandwidth by avoiding periodic polling.
- **Real-Time Feedback:** Developers get faster feedback on code changes.

What is Poll SCM?

Poll SCM is a Jenkins feature used to periodically check a Source Code Management (SCM) repository (like Git) for changes. If any changes are detected, Jenkins triggers a build. Unlike webhooks (which push changes to Jenkins in real time), Poll SCM uses a schedule-based mechanism to pull updates from the repository.

How Does Poll SCM Work?

1. Jenkins queries the repository at regular intervals (as per the configured schedule).
2. It checks for new commits, branch updates, or tags.
3. If a change is found, Jenkins triggers the associated job or pipeline.

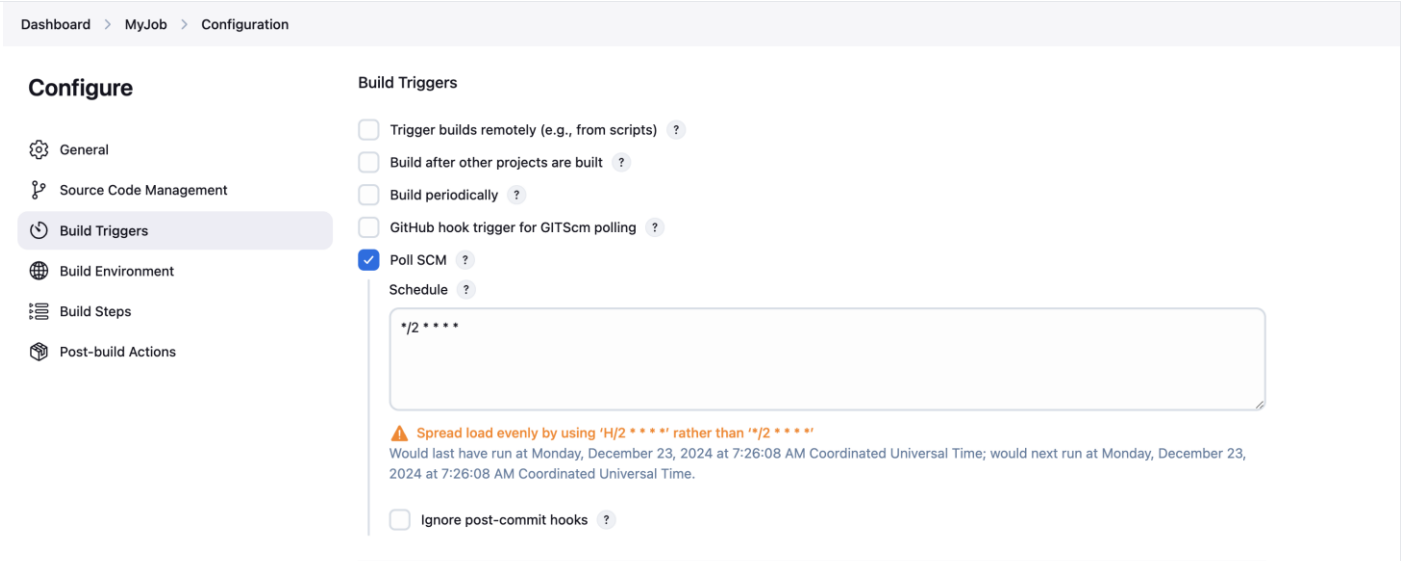
Advantages of Poll SCM:

- **Simple Setup:** No external configurations like webhooks are needed.
- **Supports All SCMs:** Works with any source code management system supported by Jenkins.

How to Configure Poll SCM in Jenkins

Step 1: Configure Jenkins Job

1. Go to the Jenkins dashboard and open the job configuration page.
2. Scroll to the **Build Triggers** section.
3. Select the checkbox for **Poll SCM**.
4. In the **Schedule** field, define the polling schedule using Cron syntax.



I have scheduled the job for every 2nd minute using Cron syntax, Now Jenkins automatically check for every 2 minutes weather any changes are available on GitHub or not. If Jenkins found new commits then it will trigger the jobs.

What is Cron Syntax?

Cron syntax is a format used to define schedules for recurring tasks. In Jenkins, it is used to schedule tasks like polling a source code repository or triggering builds at specific intervals.

A Cron expression consists of **five fields** that specify the time and frequency of execution:

Field	Allowed Values	Description
MINUTE	0-59	Minute of the hour
HOUR	0-23	Hour of the day
DOM	1-31	Day of the month
MONTH	1-12 (or names)	Month (e.g., 1 = January, 2 = February)
DOW	0-7 (or names)	Day of the week (0/7 = Sunday)

- Use * to represent "every value."
- Use H for hashed values (randomized to distribute workload).

Examples of Cron Syntax in Jenkins

1. **Every 5 minutes:**

H/5 ***

2. Every day at midnight:

00 ***

3. Every Monday at 9:00 AM:

09 ** 1

4. Every 15 minutes during office hours (9 AM to 5 PM):

H/15 9-17 ***

5. On the 1st and 15th of every month at 6:00 PM:

0 18 1,15 **

6. At 3:30 PM on weekdays:

30 15 ** 1-5

What is Build Periodically in Jenkins?

Build Periodically is a feature in Jenkins that allows you to schedule builds at regular intervals without relying on external events like code changes in the repository or webhook triggers. It uses a **cron-like syntax** to define the schedule, enabling automated and recurring builds at specific times.

Key Use Cases of Build Periodically:

1. **Scheduled Jobs:** Run nightly builds or periodic tasks, such as regression tests, backups, or report generation.
2. **Environment Maintenance:** Schedule regular updates for infrastructure or deployment pipelines.
3. **Testing Pipelines:** Continuously verify your builds and deployments even when there are no code changes.

How to Integrate Build Periodically with Jenkins

Step 1: Configure Jenkins Job

1. Go to the Jenkins dashboard and open the job configuration page.
2. Scroll to the **Build Triggers** section.
3. Select the checkbox for **Build periodically**.

Step 3: Define the Schedule

- In the **Schedule** text box, specify the frequency using cron-like syntax.

Dashboard > MyJob > Configuration

Configure

General

Source Code Management

Build Triggers

Build Environment

Build Steps

Post-build Actions

Build Triggers

☐ Trigger builds remotely (e.g., from scripts) ?

☐ Build after other projects are built ?

☒ Build periodically ?

Schedule ?

⚠ Do you really mean "every minute" when you say "*****"? Perhaps you meant "H*****" to poll once per hour

Would last have run at Monday, December 23, 2024 at 7:37:44 AM Coordinated Universal Time; would next run at Monday, December 23, 2024 at 7:37:44 AM Coordinated Universal Time.

☐ GitHub hook trigger for GITScm polling ?

☐ Poll SCM ?

Build Environment

save the **job**.

Dashboard > MyJob >

Status

</> Changes

Workspace

Build Now

Configure

Delete Project

Rename

MyJob

Permalinks

Last build (#6), 47 sec ago

Last stable build (#6), 47 sec ago

Last successful build (#6), 47 sec ago

Last completed build (#6), 47 sec ago

Builds

Filter /

Today

#7 07:42

#6 07:41

#5 07:40

#4 07:39

Now Jenkins will trigger the jobs automatically, it is independent of code i.e.. even if we will not make any changes in code, it will trigger the jobs based on the schedule.

Difference b/w Webhooks, Build Periodically & Poll SCM

Scenario: Automating Builds for a GitHub Repository

1. Webhooks:

- A developer pushes code to the `main` branch.
- GitHub sends a webhook request to Jenkins, triggering an immediate build.
- Ideal for active development teams.

2. Poll SCM:

- Jenkins checks the repository every 5 minutes.
- If changes are detected in the `main` branch, it triggers a build.
- Slight delay but works even if webhooks are unavailable.

3. Build Periodically:

- Jenkins triggers a build every night at midnight.
- Useful for nightly regression testing or scheduled tasks unrelated to code changes.

Conclusion

Integrating Git with Jenkins is a game-changer for modern software development, enabling teams to streamline CI workflows. By following this guide, you

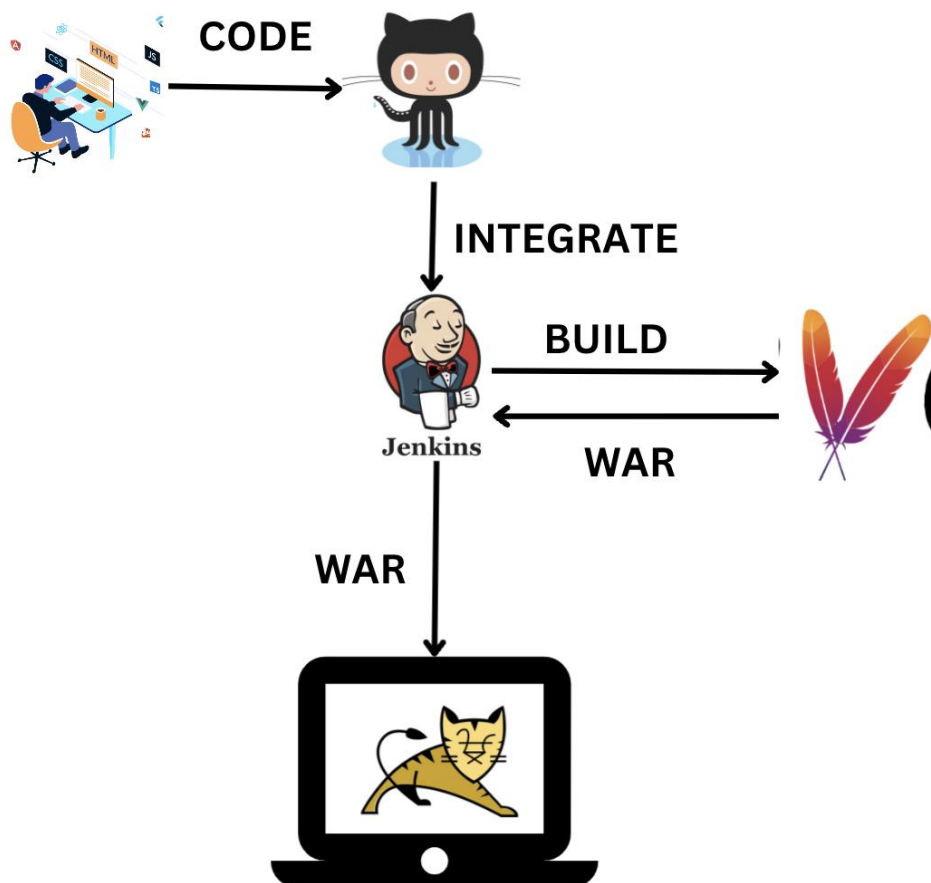
Deploy a Java Application using Jenkins

Introduction

In today's fast-paced software development world, Continuous Integration and Continuous Deployment (CI/CD) streamline application delivery. In this blog, we'll discuss how to deploy a Java-based application to an Apache Tomcat server using a CI/CD pipeline. We'll use Jenkins for automation, Maven for build management, and Git for version control.

Understanding the Tools

1. **Git:** Stores your application code and tracks changes.
2. **Maven:** Builds and manages Java projects by handling dependencies.
3. **Jenkins:** Automates the CI/CD process by integrating code building, testing, and deployment.
4. **Tomcat:** A popular server for hosting Java-based applications.



Real-Time Scenario

Imagine a software team working on an e-commerce web application. Every time a developer updates the product catalog, they push the changes to GitHub. Jenkins automatically:

1. Pulls the updated code.
2. Builds the application using Maven.
3. Perform the unit test using Maven.
4. Deploys the new version to Tomcat.

This process minimizes manual effort and ensures the application is always up-to-date.

Steps to Deploy a Java-Based Application Using CI/CD

STEP-1: Launch 2 EC2 Instances (Kernel 5.10)

- Jenkins Server
- Tomcat Server

Instances (2)

Info

Last updated
less than a minute ago

Connect

Instance state ▾

Actions ▾

Launch instances ▾

Find Instance by attribute or tag (case-sensitive)

All states ▾

< 1 >

☐	Name	Instance ID	Instance state		Instance type	Status check	Alarm status
☐	Jenkins	i-0b950cb28d3b5d0c9	Running		t2.micro	Initializing	View alarms +
☐	Tomcat	i-0aa48f381b89f683b	Running		t2.micro	Initializing	View alarms +

STEP-2: Connect Jenkins server and Install Jenkins & GIT

Copy

```
#STEP-1: INSTALLING GIT
```

```
yum install git -y
```

```
#STEP-2: GETTING THE REPO (jenkins.io --> download --> redhat)
```

```
sudo wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat-stable/jenkins.repo
```

```
sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
```

```
#STEP-3: DOWNLOAD JAVA17 AND JENKINS
```

```
yum install java-17-amazon-corretto -y
```

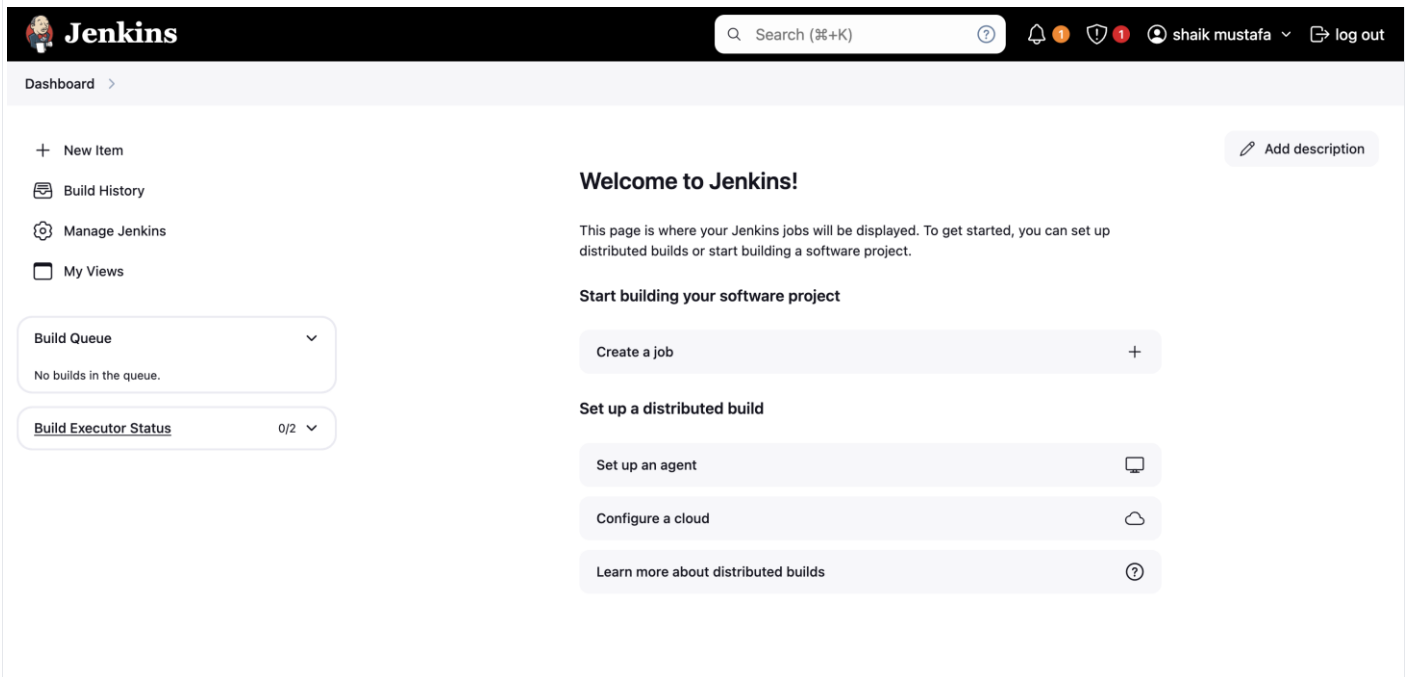
```
yum install jenkins -y
```

```
#STEP-4: RESTARTING JENKINS (when we download service it will on stopped state)
```

```
systemctl start jenkins.service
```

```
systemctl status jenkins.service
```

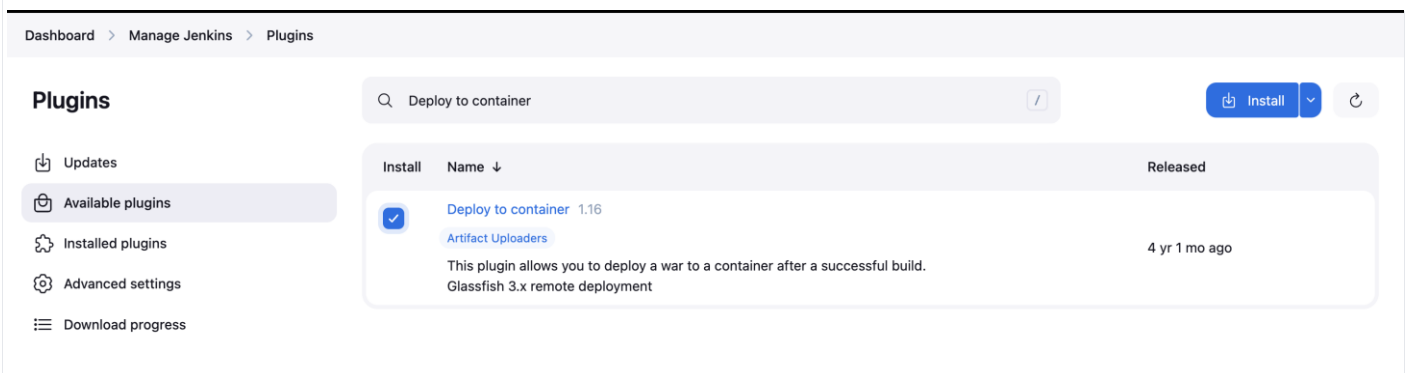
Now copy the **public-ip** of your system and access it with **8080** port.



The screenshot shows the Jenkins dashboard. At the top, there's a header with the Jenkins logo, a search bar, and user information (shaik mustafa). Below the header, the 'Dashboard' section is visible. On the left, there's a sidebar with links: '+ New Item', 'Build History', 'Manage Jenkins', and 'My Views'. The main content area has a 'Welcome to Jenkins!' message, followed by instructions on how to get started. There are two main sections: 'Start building your software project' with a 'Create a job' button, and 'Set up a distributed build' with buttons for 'Set up an agent', 'Configure a cloud', and 'Learn more about distributed builds'. A 'Build Queue' widget shows 'No builds in the queue.' and a 'Build Executor Status' widget shows '0/2'.

STEP-3: Now Install **Deploy to container** plugin

Go to Manage Jenkins » Plugins » Available Plugins » Deploy to container



The screenshot shows the 'Plugins' page in Jenkins. The breadcrumb trail is 'Dashboard > Manage Jenkins > Plugins'. The left sidebar has links: 'Updates', 'Available plugins' (selected), 'Installed plugins', 'Advanced settings', and 'Download progress'. The main content area has a search bar with 'Deploy to container' entered. Below the search bar, there's a table of plugins. The first plugin listed is 'Deploy to container 1.16', which is marked as installed with a blue checkmark. Below the plugin name, it says 'Artifact Uploaders'. The description states: 'This plugin allows you to deploy a war to a container after a successful build. Glassfish 3.x remote deployment'. The 'Released' column shows '4 yr 1 mo ago'. There are 'Install', 'Uninstall', and 'Refresh' buttons at the top right of the plugin list.

STEP-4: Integrate Maven

Go to Manage Jenkins » tools and select maven

under the **Maven Installations** click on **Add Maven**

Name: **mymaven**

Version: **default (3.9.9)**

Dashboard > Manage Jenkins > Tools

Maven installations

Add Maven

≡ Maven

Name

mymaven

☒ Install automatically ?

≡ Install from Apache

Version

3.9.9

Add Installer ▾

Add Maven

Save Apply

Now click on **save**

STEP-5: Create a Free Style Job

 **Jenkins**     shaik mustafa ▾  log out

Dashboard > New Item

New Item

Enter an item name

MyDeployment

Select an item type

 **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

 **Pipeline**
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.

 **Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

 **Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

OK

Go to **Source Code Management** and select **Git** and enter the repo-url (<https://github.com/usubbu/one.git>)

Dashboard > MyDeployment > Configuration

Configure

General

Source Code Management

Build Triggers

Build Environment

Build Steps

Post-build Actions

☐ Invoke builds ?

☐ Execute concurrent builds if necessary ?

Advanced ▾

Source Code Management

☐ None

☒ Git ?

Repositories ?

Repository URL ?

https://github.com/usubbu/one.git

Credentials ?

- none - ▾

+ Add

Advanced ▾

Go to **Build Steps** and select **Invoke top-level Maven targets** and select the Maven version as **mymaven**

Now enter the goal as **clean package**

Dashboard > MyDeployment > Configuration

Configure

General

Source Code Management

Build Triggers

Build Environment

Build Steps

Post-build Actions

☐ Delete workspace before build starts

☐ Use secret text(s) or file(s) ?

☐ Add timestamps to the Console Output

☐ Inspect build log for published build scans

☐ Terminate a build if it's stuck

☐ With Ant ?

Build Steps

≡ Invoke top-level Maven targets ?

Maven Version

mymaven ▾

Goals

clean package ▾

Advanced ▾

Add build step ▾

Save

Apply

Save the job and click on **Build Now**

Dashboard > MyDeployment >

Status

</> Changes

Workspace

Build Now

Configure

Delete Project

Rename

Builds

Today

✓ #1 18:29

MyDeployment

Permalinks

If the build gets success, go to `workspace` and open the `target` folder, we will get `war` file

Dashboard > MyDeployment > Workspace of MyDeployment on Built-In Node

Status

</> Changes

Workspace

Build Now

Configure

Delete Project

Rename

Builds

Filter

Today

✓ #1 18:29

Workspace of MyDeployment on Built-In Node

MyDeployment / target /

maven-archiver

myweb-8.6.7

myweb-8.6.7.war 24 Dec 2024, 18:29:58 1.96 KiB

(all files in zip)

STEP-6: Deploy the application on Tomcat Server

connect the tomcat server and run the script to setup tomcat

Run the below script to setup tomcat

Copy

```
amazon-linux-extras install java-openjdk11 -y
```

```
wget https://dlcdn.apache.org/tomcat/tomcat-9/v9.0.102/bin/apache-tomcat-9.0.102.tar.gz
```

```
tar -zxvf apache-tomcat-9.0.102.tar.gz
```

```
sed -i '56 a<role rolename="manager-gui"/>' apache-tomcat-9.0.102/conf/tomcat-users.xml
```

```
sed -i '57 a<role rolename="manager-script"/>' apache-tomcat-9.0.102/conf/tomcat-users.xml
```

```
sed -i '58 a<user username="tomcat" password="admin@123" roles="manager-gui, manager-script"/>' apache-tomcat-9.0.102/conf/tomcat-users.xml
```

```
sed -i '59 a</tomcat-users>' apache-tomcat-9.0.102/conf/tomcat-users.xml
```

```
sed -i '56d' apache-tomcat-9.0.102/conf/tomcat-users.xml
```

```
sed -i '21d' apache-tomcat-9.0.102/webapps/manager/META-INF/context.xml
```

```
sed -i '22d' apache-tomcat-9.0.102/webapps/manager/META-INF/context.xml
```

```
sh apache-tomcat-9.0.102/bin/startup.sh
```

Remember the tomcat credentials

- username**:* tomcat
- password**:* admin@123

If you wish to modify, change the credentials on above script also.

Now access the tomcat dashboard with **public-ip** of tomcat server with **8080** port.

Now go to **Manager App** and enter the credentials, You will get the **Tomcat Web Application Manager** dashboard.

← → ↻

Not Secure 54.88.54.240:8080/manager/html

☆

⋮



Tomcat Web Application Manager

Message:

OK

Manager

List Applications

HTML Manager Help

Manager Help

Server Status

Applications

Path	Version	Display Name	Running	Sessions	Commands
/	None specified	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	None specified	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	None specified	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager	None specified	Tomcat Host Manager Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager	None specified	Tomcat Manager Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes

STEP-7: Integrate Tomcat with Jenkins

Configure the Jenkins job and select **Post Build Actions** and select **Deploy war/ear to a container**

WAR/EAR file: **target/*.war**

context path: **swiggy**

Dashboard > MyDeployment > Configuration

Configure

General

Source Code Management

Build Triggers

Build Environment

Build Steps

Post-build Actions

Post-build Actions

Deploy war/ear to a container

WAR/EAR files ?
target/*.war

Context path ?
swiggy

Containers
Add Container ▾

☐ Deploy on failure

Add post-build action ▾

Save Apply

REST API Jenkins 2.479.2

and click on **add container** and select **Tomcat 9.x Remote**

Dashboard > MyDeployment > Configuration

Configure

- General
- Source Code Management
- Build Triggers
- Build Environment
- Build Steps
- Post-build Actions

Containers

Tomcat 9.x Remote

Credentials

- none -

+ Add

Tomcat URL ?

Advanced

Add Container

☐ Deploy on failure

Add post-build action

Save Apply

Now add the tomcat credentials, click on **Add** select **Jenkins**

Jenkins Credentials Provider: Jenkins

Kind

Username with password

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

Username ?

tomcat

☐ Treat username as secret ?

Password ?

.....

ID ?

appserver

Description ?

tomcat credentials

Save Apply

Now click on **Add** and select the **credentials** and enter the **tomcat url**

Dashboard > MyDeployment > Configuration

Configure

- General
- Source Code Management
- Build Triggers
- Build Environment
- Build Steps
- Post-build Actions

Containers

Tomcat 9.x Remote

Credentials
tomcat/***** (tomcat credentials)

+ Add

Tomcat URL ?
http://54.88.54.240:8080

Advanced

Add Container

☐ Deploy on failure

Add post-build action

Save Apply

Now save the job and click on **Build Now**, If the build gets success then our application will be deployed on tomcat successfully.

 **Jenkins** Search

Dashboard > MyDeployment >

Status

</> Changes

Workspace

Build Now

Configure

Delete Project

Rename

Builds

Filter

Today

#2 18:48

#1 18:29

MyDeployment

Permalinks

- Last build (#1), 17 min ago
- Last stable build (#1), 17 min ago
- Last successful build (#1), 17 min ago
- Last completed build (#1), 17 min ago

The build is **success**, Now lets **refresh** the **tomcat** page and we will see **swiggy** app is **running**

← → ↻

Not Secure 54.88.54.240:8080/manager/html

☆ 👤 ⋮



Tomcat Web Application Manager

Message: OK

Manager

[List Applications](#)[HTML Manager Help](#)[Manager Help](#)[Server Status](#)

Applications

Path	Version	Display Name	Running	Sessions	Commands
/	None specified	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	None specified	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	None specified	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager	None specified	Tomcat Host Manager Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager	None specified	Tomcat Manager Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/swiggv	None specified	Archetype Created Web Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes

Open the swiggv app and check the output.

← → ↻

Not Secure 54.88.54.240:8080/swiggv/?#projects

☆ 👤 ⋮

Mustafa - DevOps Engineer

Building Automation & Scalable Solutions for Modern Applications

About Skills Projects Contact

About Me

Hello! I'm Mustafa, a DevOps Engineer passionate about optimizing workflows and delivering scalable, automated solutions. I specialize in tools like Jenkins, Docker, Kubernetes, Terraform, and cloud platforms such as AWS and Azure.

Skills

CI/CD - Jenkins, GitHub Actions

Containerization - Docker

Orchestration - Kubernetes

Infrastructure as Code - Terraform

Cloud Platforms - AWS, Azure

Monitoring - Prometheus, Grafana

To automate the delivery process, we can also implement Webhooks!.

Best Practices

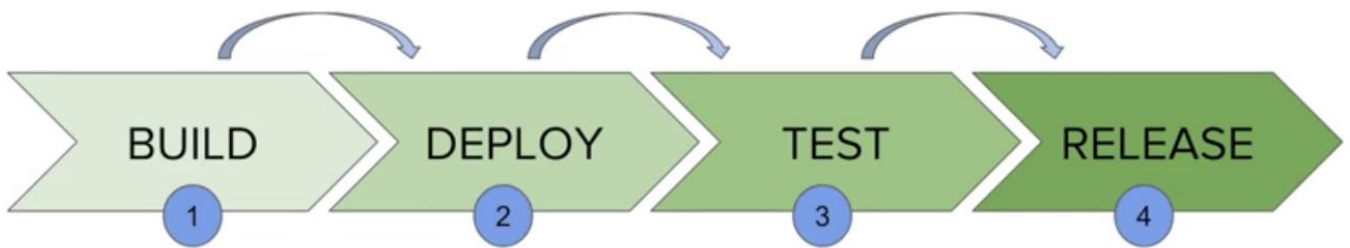
- Use a separate staging environment for testing deployments before pushing to production.
- Automate testing as part of the CI/CD pipeline to catch bugs early.

- Secure your Jenkins and Tomcat configurations to prevent unauthorized access.

Conclusion

By automating deployment with CI/CD, you save time, reduce errors, and deliver features faster. Deploying Java applications to Tomcat with tools like Jenkins and Maven is straightforward when done step-by-step. Start building your pipeline today to experience the power of automation.

Introducing the power of pipelines in Jenkins



Now its time to explore the pipeline jobs in Jenkins

Software development moves fast, and keeping up means delivering updates quickly and reliably. Continuous Integration (CI) and Continuous Delivery (CD) are practices that help teams stay agile and produce high-quality applications efficiently. Jenkins, a top tool for CI/CD, simplifies this by automating workflows. One of its standout features is pipelines, which make these processes smoother and more adaptable. Let's explore Jenkins pipelines with simple examples to see how they can make your work easier.

What Are Jenkins Pipelines?

A Jenkins pipeline is a way to describe your entire CI/CD process as code. Think of it as a script that automates steps like building, testing, and deploying your application. For example, instead of manually running tests after every change, a pipeline ensures it happens automatically.

Jenkins pipelines can be written in two ways:

- **Declarative Pipelines:** These are simpler and easier to read, perfect for straightforward workflows.
- **Scripted Pipelines:** These offer more flexibility, suitable for complex setups.

Key Benefits of Jenkins Pipelines

1. **Automation:**

- Pipelines let you automate tasks, saving time and reducing errors. For instance, you can automatically deploy an app to a test server after a successful build.

2. **Version Control:**

- Pipeline scripts can be stored in the same repository as your code, ensuring changes are tracked and collaborative.

3. **Error Detection:**

- Pipelines provide detailed logs for each stage, helping you quickly identify and fix issues.

4. **Extensibility:**

- Jenkins pipelines integrate with popular tools like Docker, Kubernetes, and GitHub.

Declarative Pipeline Syntax:

Copy

```
pipeline {  
    agent any  
    stages {  
        stage('Code') {  
            steps {  
                echo 'Getting the project code'  
            }  
        }  
        stage('Build') {  
            steps {  
                echo 'Build the code'  
            }  
        }  
        stage('Deploy') {  
            steps {  
                echo 'Deploy the code.'  
            }  
        }  
    }  
}
```

```
}
```

```
}
```

```
}
```

```
}
```

Scripted Pipeline Syntax:

Copy

```
node {
```

```
    stage('Code') {
```

```
        echo 'this is stage-1'
```

```
    }
```

```
    stage('Build') {
```

```
        echo 'this is build stage.'
```

```
    }
```

```
    stage('Deploy') {
```

```
        echo 'this is deploy.'
```

```
    }
```

```
}
```

Difference Between Freestyle and Pipeline Jobs

1. **Freestyle Jobs:**

- Simple configurations without coding.
- Example: A job that checks out code from GitHub and builds it.

- Limitations: Hard to manage complex workflows.

2. Pipeline Jobs:

- Defined as code, stored in a Jenkins file.
- Flexible, scalable, and better suited for modern DevOps practices.
- Example: Automating a full CI/CD process with stages for build, test, and deploy.

In the next blog, I am going to reveal the fundamentals of pipelines.

Conclusion

Jenkins pipelines simplify and enhance CI/CD processes. Whether you're a developer building an app or an operations engineer managing deployments, pipelines provide consistency and scalability. Start using Jenkins pipelines today to save time, reduce errors, and deliver value faster.

How to write your first pipeline in Jenkins file

Jenkins file:

A **Jenkins file** is a text file that contains the definition of a Jenkins pipeline. It allows you to automate the steps in your Continuous Integration and Continuous Delivery (CI/CD) process by scripting them in Groovy. The Jenkins file is version-controlled alongside your project, ensuring transparency and consistency in your pipeline configuration.

Declarative Pipeline Syntax:

Copy

Copy

```
pipeline {  
    agent any  
    stages {  
        stage('Stage-Name') {  
            steps {  
                echo "This is my first pipeline"  
            }  
        }  
    }  
}
```

```
}
```

```
}
```

```
}
```

Pipeline Structure and Components

1. `pipeline` Block

The top-level block wraps the entire Jenkins file and defines the pipeline. It specifies the environment and stages of the process.

2. `agent any`

- **Purpose:** Specifies where the pipeline runs.
- `any`: The pipeline can execute on any available agent (node) in the Jenkins setup. This provides flexibility if there are multiple agents in the environment. By default it runs on master server.

Stages

3. `stage('Code')`

- **Purpose:** Represents the Build phase of the pipeline.
- **steps:**
 - `echo 'this is my first pipeline'`: Outputs a message to the Jenkins console log, simulating the build step.
 - In a real pipeline, this step might include commands to compile code or install dependencies.

Real-time pipelines to get Source Code from GitHub:

This pipeline is used to get the code from GitHub to CI server.

Copy

Copy

```
pipeline {
```

```
    agent any
```

```
    stages {
```

```
        stage('Code') {
```

```
steps {
```

```
git branch: "main", credentialsId: "github", url: 'https://github.com/devops0014/one.git'
```

```
}
```

```
}
```

```
}
```

```
}
```

Multi Stage pipeline:

This pipeline contains multiple stages like code, build, test and deploy.

Copy

Copy

```
pipeline {
```

```
agent any
```

```
stages {
```

```
stage('Hello') {
```

```
steps {
```

```
echo 'Hello World'
```

```
}
```

```
}
```

```
stage('Test') {
```

```
steps {
```

```
echo 'Hello World test'
```



```

    }

}

stage('Deploy') {

    steps {

        echo 'Hello World deploy'

    }

}

}

}

}

}

```

Pipeline as a Code:'

This pipeline is used to execute the commands.

Copy

Copy

```

pipeline {

    agent any

    stages {

        stage('CMD') {

            steps {

                sh 'touch file1'

                sh 'pwd'

```

```
}
```

```
}
```

```
}
```

```
}
```

This pipeline is used to execute multiple commands at a time:

Copy

Copy

```
pipeline {
```

```
  agent any
```

```
  stages {
```

```
    stage('CMD') {
```

```
      steps {
```

```
        sh '''
```

```
          touch file2
```

```
          pwd
```

```
          date
```

```
          whoami
```

```
        '''
```

```
      }
```

```
    }
```

```
  }
```

```
}
```

Pipeline with variables:

This pipeline is used to define the variables.

Copy

Copy

```
pipeline {  
  
    agent any  
  
    environment {  
  
        name = 'Mustafa'  
  
    }  
  
    stages {  
  
        stage('ENV') {  
  
            steps {  
  
                echo "My name is $name"  
  
            }  
  
        }  
  
    }  
  
}
```

This pipeline is used to print the build ID's

Copy

Copy

```
pipeline {  
  
    agent any
```

```
stages {  
  
  stage('ENV') {  
  
    steps {  
  
      sh 'echo "${BUILD_ID}"'  
  
    }  
  
  }  
  
}
```

In this pipeline, we can see local and global variables.

Copy

Copy

```
pipeline {  
  
  agent any  
  
  environment {  
  
    name = "Mustafa"  
  
  }  
  
  stages {  
  
    stage('one') {  
  
      steps {  
  
        echo "My name is $name"  
  
      }  
  
    }  
  
    stage('two') {
```

```
environment {
```

```
    course = "DevOps"
```

```
}
```

```
steps {
```

```
    echo "$name is a $course trainer"
```

```
}
```

```
}
```

```
}
```

```
}
```

In the above pipeline, we can see `name = mustafa` is a global variable, we can access this variable on any stage in the pipeline. `course = DevOps` is a local variable, we can use this only on stage-2.

NOTE:

If you want to access a variable on all pipelines, go to manage jenkins » system » select variables option and pass the values.

Jenkins Pipeline to deploy Java application | Git, Maven, Nexus & Tomcat

my pipeline contains 4 stages [code, build & Unit Test, Artifact, deploy]

Tools we used for each stage:

- code: GIT
- Build: Maven
- Unit Test: Maven
- Artifact: Nexus
- Deploy: Tomcat



CODE

BUILD & TEST

ARTIFACT

DEPLOY

STEP BY STEP PROCESS TO DEPLOY THE APPLICATION:

STEP-1: Launch 3 Instances

- Jenkins
- Nexus
- Tomcat

STEP-2: refer the below scripts to install Jenkins, Tomcat and Nexus on respective servers.

Jenkins Script:

Copy

```
#STEP-1: INSTALLING GIT
```

```
yum install git -y
```

```
#STEP-2: GETTING THE REPO (jenkins.io --> download --> redhat)
```

```
sudo wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat-stable/jenkins.repo
```

```
sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
```

```
#STEP-3: DOWNLOAD JAVA17 AND JENKINS
```

```
yum install java-17-amazon-corretto -y
```

```
yum install jenkins -y
```

```
#STEP-4: RESTARTING JENKINS (when we download service it will on stopped state)
```

```
systemctl start jenkins.service
```

```
systemctl status jenkins.service
```

Tomcat Script:

Copy

```
yum install java-17-amazon-corretto -y
```

```
wget https://dlcdn.apache.org/tomcat/tomcat-9/v9.0.98/bin/apache-tomcat-9.0.98.tar.gz
```

```
tar -zxvf apache-tomcat-9.0.98.tar.gz
```

```
sed -i '56 a\<role rolename="manager-gui"/>' apache-tomcat-9.0.98/conf/tomcat-users.xml
```

```
sed -i '57 a\<role rolename="manager-script"/>' apache-tomcat-9.0.98/conf/tomcat-users.xml
```

```
sed -i '58 a\<user username="tomcat" password="admin@123" roles="manager-gui, manager-script"/>' apache-tomcat-9.0.98/conf/tomcat-users.xml
```

```
sed -i '59 a\</tomcat-users>' apache-tomcat-9.0.98/conf/tomcat-users.xml
```

```
sed -i '56d' apache-tomcat-9.0.98/conf/tomcat-users.xml
```

```
sed -i '21d' apache-tomcat-9.0.98/webapps/manager/META-INF/context.xml
```

```
sed -i '22d' apache-tomcat-9.0.98/webapps/manager/META-INF/context.xml
```

```
sh apache-tomcat-9.0.98/bin/startup.sh
```

Nexus Script:

Copy

```
yum install java-17-amazon-corretto -y
```

```
mkdir /app
```

```
cd /app
```

```
wget -O nexus.tar.gz https://download.sonatype.com/nexus/3/latest-unix.tar.gz
```

```
tar -zxvf nexus.tar.gz
```

```
mv nexus-* nexus
```

```
useradd nexus
```

```
chown -R nexus:nexus *
```

```
vim /app/nexus/bin/nexus.rc
```

```
sed -i '1s/.*/run_as_user="nexus"/' /app/nexus/bin/nexus.rc
```

```
./nexus/bin/nexus start
```

```
./nexus/bin/nexus status
```

STEP-3: Install the following plugins on Jenkins dashborad

- Nexus Artifact Uploader
- Deploy to Container

STEP-4: Create a Pipeline job and write the below Jenkinsfile

Copy

```
pipeline {
```

```
    agent any
```

```
    tools {
```

```
        maven "mymaven"
```

```
    }
```

```
    stages {
```

```
        stage('Code') {
```

```
            steps {
```

```
                git 'https://github.com/devops0014/one.git'
```

```
            }
```



```

    }

    stage ("Build") {

        steps {

            sh 'mvn clean package'

        }

    }

    stage ("Artifact") {

        steps {

            nexusArtifactUploader artifacts: [[artifactId: 'myweb', classifier: '', file: 'target/myweb-8.6.9.war', type: 'war']], credentialsId:
            'nexus', groupId: 'in.javahome', nexusUrl: '54.227.157.176:8081', nexusVersion: 'nexus3', protocol: 'http', repository: 'myrepo', version:
            '8.6.9'

        }

    }

    stage ("Deploy") {

        steps {

            deploy adapters: [tomcat9(credentialsId: 'tomcat', path: '', url: 'http://54.156.87.218:8080')], contextPath: 'myapp', war:
            'target/*.war'

        }

    }

}

```

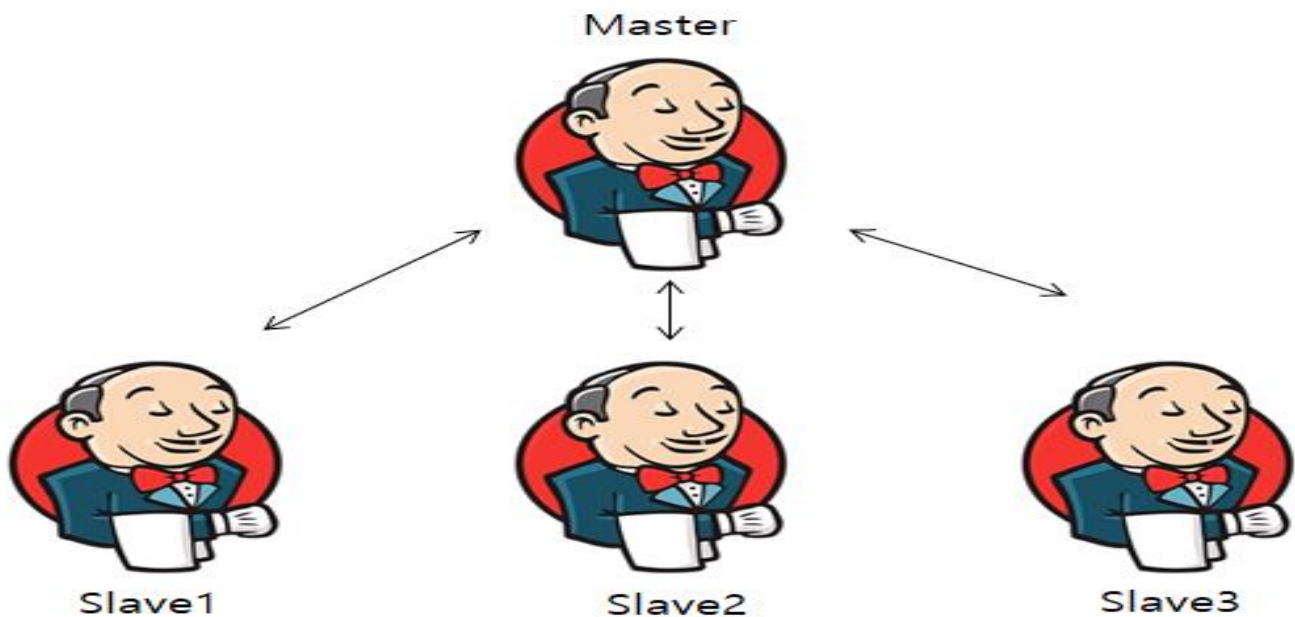
NOTE: use pipeline syntax for getting the scrips for each stage.

The Power of the Master-Slave Architecture

In the world of continuous integration and delivery (CI/CD), Jenkins stands as one of the most popular automation tools. One of the key features that make Jenkins powerful and scalable is its master-slave architecture. In this blog, we will delve into the concept, explain how it works, and provide real-world scenarios to make it easier to understand.

What is the Master-Slave Architecture in Jenkins?

The master-slave architecture is a distributed system setup in Jenkins where a **master** node delegates work to one or more **slave** nodes. The master node is the central controller, while the slave nodes are worker machines that perform the actual tasks. This design allows Jenkins to handle large and complex build processes by distributing the load across multiple machines.



Components of the Architecture

1. Master Node:

- Responsible for scheduling jobs.
- Maintains the build configurations, plugins, and user interfaces.
- Delegates tasks to slave nodes when necessary.

2. Slave Node:

- A worker machine that performs tasks assigned by the master.
- Runs build jobs in its environment (e.g., Linux, Windows, macOS).
- Communicates with the master to send job statuses and logs.

How Does It Work?

When a build is triggered on the Jenkins master, it evaluates where the job can best be executed based on resource availability, node labels, or specific job configurations. If the job requires a specific environment, it will delegate the task to a slave node configured for that purpose.

Real-World Scenarios

Scenario 1: Cross-Platform Builds

Imagine a software product that needs to run on multiple platforms: Windows, Linux, and macOS. You can configure Jenkins with a master node and three slave nodes, each running a different operating system. When a build is triggered, Jenkins distributes the workload to the appropriate slave based on the target platform.

For example:

- Windows builds run on the Windows slave.
- Linux builds run on the Linux slave.
- macOS builds run on the macOS slave.

This setup ensures that you can test and build your software in multiple environments simultaneously, saving time and resources.

Scenario 2: High-Volume Builds

A large e-commerce company has multiple development teams working on various microservices. Each team triggers builds frequently, causing a bottleneck when all jobs are queued on a single master node. By setting up multiple slave nodes, Jenkins can distribute these builds across the slaves, significantly reducing build times.

For instance:

- Microservice A's build runs on Slave 1.
- Microservice B's build runs on Slave 2.
- Microservice C's build runs on Slave 3.

Advantages of the Master-Slave Architecture

- **Scalability:** Add more slave nodes to handle increased workloads.
- **Flexibility:** Configure slaves with specific environments or tools.
- **Efficiency:** Parallel execution reduces build times.
- **Fault Tolerance:** If a slave fails, the master can redistribute tasks to other available slaves.

Setting Up a Master-Slave Configuration in Jenkins:

STEP-1: Launch 2 instances with key-pair (Master, Slave)

Instances (2) Info

Last updated 14 minutes ago

Refresh

Connect

Instance state

Actions

Launch instances

Find Instance by attribute or tag (case-sensitive)

All states

< 1 >

Settings

☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availa
☐	MASTER	i-0e4147417be788d9e	Running	t2.micro	Initializing	View alarms +	ap-sou
☐	SLAVE	i-075e5da486c9b13d1	Running	t2.micro	Initializing	View alarms +	ap-sou

STEP-2: Install Jenkins on Master

Copy

#STEP-1: INSTALLING GIT

```
yum install git -y
```

#STEP-2: GETTING THE REPO (jenkins.io --> download --> redhat)

```
sudo wget -O /etc/yum.repos.d/jenkins.repo \
```

```
https://pkg.jenkins.io/redhat-stable/jenkins.repo
```

```
sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
```

#STEP-3: DOWNLOAD JAVA11 AND JENKINS

```
yum install java-17-amazon-corretto -y
```

```
yum install jenkins -y
```

#STEP-4: RESTARTING JENKINS (when we download service it will on stopped state)

```
systemctl start jenkins.service
```

```
systemctl status jenkins.service
```

 Jenkins

Search (⌘+K)



 1

 1

shaik mustafa

log out

Dashboard

+ New Item

 Build History

 Manage Jenkins

 My Views

Build Queue

No builds in the queue.

Build Executor Status

0/2

Add description

Welcome to Jenkins!

This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.

Start building your software project

Create a job

Set up a distributed build

Set up an agent

Configure a cloud

Learn more about distributed builds

STEP-3: Install Java17 on Slave Server

Copy

```
yum install java-17-amazon-corretto -y
```

STEP-4: Go to Jenkins Dashboard » Manage Jenkins » Nodes

 Jenkins

Search (⌘+K)



 1

 1

shaik mustafa

log out

Dashboard > Manage Jenkins > Nodes

 Nodes

 Clouds

Build Queue

No builds in the queue.

Build Executor Status

0/2

Nodes

+ New Node

Configure Monitors



S	Name ↓	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	Response Time
	Built-In Node	Linux (amd64)	In sync	5.67 GiB	 0 B	5.67 GiB	0ms
	last checked	3 min 5 sec	3 min 5 sec	3 min 5 sec	3 min 5 sec	3 min 5 sec	3 min 5 sec

Icon: S M L

Legend

Click on **New Node**

New node

Node name

slave1

Type



Permanent Agent

Adds a plain, permanent agent to Jenkins. This is called "permanent" because Jenkins doesn't provide higher level of integration with these agents, such as dynamic provisioning. Select this type if no other agent types apply — for example such as when you are adding a physical computer, virtual machines managed outside Jenkins, etc.

Create

Name ?

slave1

Description ?

Plain text [Preview](#)

Number of executors ?

2

Remote root directory ?

/home/ec2-user/jenkins

Labels ?

dev

Labels ?

dev

Usage ?

Only build jobs with label expressions matching this node

Launch method ?

Launch agents via SSH

Host ?

172.31.1.100

Credentials ?

- none -

+ Add

 The selected credentials cannot be found

Host: **Slave-server-Private-IP**

Now click on **Add credentials**

Kind: **SSH Username with Private Key**

Username: **ec2-user**

Now add the **Private Key** (PEM file)

Launch method ?

Launch agents via SSH

Host ?

172.31.1.100

Credentials ?

ec2-user (slave server password)

+ Add

Host Key Verification Strategy ?

Non verifying Verification Strategy

Advanced

Now click on **save**

Now we have implemented Master-Slave Architecture, Lets build a job and execute it on slave-server

Go to **Jenkins Dashboard** and click on **New Item**

Dashboard >

+ New Item

Build History

Project Relationship

Check File Fingerprint

Manage Jenkins

My Views

Build Queue

No builds in the queue.

Build Executor Status

Built-In Node

0/2

slave1

0/2

Welcome to Jenkins!

This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.

Start building your software project

Create a job +

Add description

Enter the Job name and select the type

New Item

Enter an item name

Job-1

Select an item type



Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Pipeline

Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.



Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.



Folder

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

OK

Now enter tyhe label name as

dev

Dashboard > Job-1 > Configuration

Configure

General

Source Code Management

Build Triggers

Build Environment

Build Steps

Post-build Actions

Description

Plain text [Preview](#)

☐ Discard old builds ?

☐ GitHub project

☐ This project is parameterised ?

☐ Throttle builds ?

☐ Execute concurrent builds if necessary ?

☒ Restrict where this project can be run ?

Label Expression ?

dev

[Label dev](#) matches 1 node. Permissions or other restrictions provided by plugins may further reduce that list.

Advanced ▾

Save and build the job, it will be executed in slave server.

Dashboard > Job-1 >

Status

Changes

Workspace

Build Now

Configure

Delete Project

Rename

Job-1

Permalinks

Builds

⋮

🔗

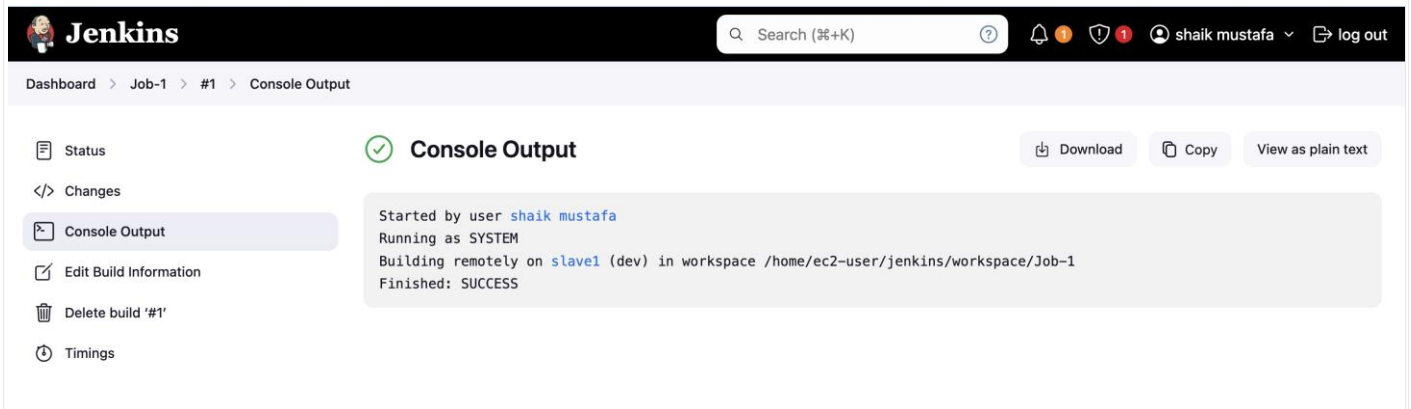
Today

✓

#1 05:21

▾

Lets Open the console output to confirm

The screenshot shows the Jenkins web interface. At the top is a black header with the Jenkins logo, a search bar, and user information. Below the header is a breadcrumb trail: Dashboard > Job-1 > #1 > Console Output. On the left is a sidebar with links: Status, Changes, Console Output (selected), Edit Build Information, Delete build '#1', and Timings. The main area is titled 'Console Output' with a green checkmark icon. It contains a log entry: 'Started by user shaik mustafa', 'Running as SYSTEM', 'Building remotely on slave1 (dev) in workspace /home/ec2-user/jenkins/workspace/Job-1', and 'Finished: SUCCESS'. To the right of the log entry are buttons for 'Download', 'Copy', and 'View as plain text'.

As you can observer, the job is executed on remotely (slave1).

Conclusion

The master-slave architecture in Jenkins is a robust solution for handling complex CI/CD workflows. By distributing tasks across multiple machines, it ensures efficient, scalable, and fault-tolerant operations. Whether you're dealing with cross-platform builds, high-volume tasks, or resource-intensive processes, this architecture empowers you to optimize your development pipeline.